全栈开发技术期末项目报告

阅记

原生电子书阅读器与 AI 阅读助手

项目名称：阅记原生电子书阅读器与 AI 阅读助手

姓名：周宇斌

学号：20246085

班级：软件 2401

2026 年 10 月 25 日

目录

项目背景与意义 3

1 需求分析 3

1.1 功能需求 3

1.2 非功能需求 4

2 系统设计 5

2.1 架构设计 5

2.2 功能模块设计 6

2.3 数据/数据库设计 7

2.4 接口与类设计 9

2.5 技术选型说明 10

3 系统实现 11

3.1 关键模块的核心实现 11

3.2 核心功能与 UI 截图 15

3.3 难点和解决方案 22

4 系统测试 23

4.1 测试环境 23

4.2 测试用例 24

4.3 测试截图 27

5 总结与展望 28

项目背景与意义

个人阅读中，书籍资料、阅读活动与摘录通常分散记录。阅记以本地桌面应用统一管理这些信息，使用户能够阅读电子书、追踪阅读活动、整理感想并恢复自己的数据。项目在原有阅读与书摘管理基础上，加入 TXT/EPUB 原生阅读器及 DeepSeek AI 阅读辅助，形成“导入—阅读—摘录—计划—统计—备份”的业务闭环。

系统同时服务两类阅读方式：纸质书等手动书籍以真实页码和分钟数记录活动；电子书以稳定正文位置、百分比和自动阅读会话记录活动。动态排版页号仅服务当前阅读界面，不作为纸质页数计入统计。AI 仅处理用户主动确认的正文范围，本地阅读和手动书摘不依赖网络。

1 需求分析

1.1 功能需求

模块	主要功能与业务结果
书籍管理	手动录入、编辑、搜索、状态筛选及分页；TXT/EPUB 多选导入、收件区扫描、摘要去重及受管副本删除。
原生阅读	章节目录、分页翻阅、键盘操作、F11 全屏；字体搜索与预览、字号和纸张配色；三面板独立收起。
进度与会话	按正文锚点续读；首屏实际呈现后启动会话，正常离开时保存时长与最后位置；完成标记与回看位置分离。
书摘与收藏	手动书摘及原生选区保存；Ctrl+C / Ctrl+S；去重、感想、收藏、编辑、删除和精确原文回跳。
AI 阅读辅助	当前页、选区和全书页范围解读；生成并自动保存通过核验的书摘；去重、本地保存重试及批次撤销。
计划与统计	制定目标日期并呈现计划状态；手动页数与两类阅读时长分别统计，电子书不生成虚假物理页数。
设置与数据	浅深主题、真实 AI 配置与会话凭据；完整 ZIP 备份恢复、旧 JSON 兼容恢复及异常操作日志恢复。

1.2 非功能需求

离线能力与可维护性

业务数据保存在当前用户本地应用数据目录。断网后可阅读已有书籍、保存手动书摘、查看计划与统计；只有用户主动确认的 AI 操作联网。首次依赖还原需要 NuGet 来源或完整缓存，不能把“日常阅读离线”理解为“首次构建无需联网”。工程保留 UI、Library 与 UnitTest 的职责划分，业务通过接口注入，以便替换平台能力和隔离测试。

一致性与可靠性

数据库访问串行，跨表写入采用事务；文件系统与数据库的组合操作采用 staging、trash 与持久化操作日志恢复。书籍使用不可变 BookUid，正文使用 DocumentKey 标识版本，维护操作更新 DataEpoch，防止旧页面或迟到响应写入新数据。恢复先完整校验并生成原库自动备份，成功后再替换数据。原始导入文件保持不变。

交互与响应

界面使用中文标签、六项侧边菜单、统一表单和浅深主题。文件选择使用系统原生对话框。正文重排保留稳定位置和仍有效的选区；耗时任务提供忙碌、错误和取消反馈。以异步命令与可取消后台索引减少界面阻塞；性能结论限于实际测试样本，具体采样口径见第 4 章。

AI 数据与费用边界

请求前需保存上传知情确认，并再次确认所选范围及可能费用。会话 Key 只在进程内存保存，不写普通配置、源码、日志或备份；退出后须重新输入。请求默认输入上限 24000 个 UTF-16 单元、输出 2048 token、超时 45 秒；输入硬上限 60000 个 UTF-16 单元，页范围最多 200 页和 2048 个片段。字符单元、token 和真实费用是不同概念，费用以供应商账单为准。

要求	系统处理
请求控制	无凭据或未同意上传时在 HTTP 前拒绝；超时、取消或错误不自动重新计费请求。
原文可信度	AI 候选须在已冻结的连续原文中逐字匹配并唯一定位；通过原文核验不代表 AI 解读正确。
范围限制	无账号登录、云同步与在线书源；不支持 PDF/OCR、DRM 或固定版式 EPUB。
平台边界	已实测 Windows 11 x64、150% 缩放；其他缩放、跨屏和商业 EPUB 未扩大验收。

2 系统设计

2.1 架构设计

解决方案包含 ReadJournal、ReadJournal.Library 和 ReadJournal.UnitTest 三个工程。ReadJournal 负责 Avalonia 视图、原生阅读表面、实际导航、布局及平台服务；Library 负责模型、业务规则、存储、阅读器生命周期、AI 工作流和备份协议，不引用 Avalonia 控件；UnitTest 使用 xUnit、Moq、Headless 视图与临时真实 SQLite 数据库验证规则和绑定。

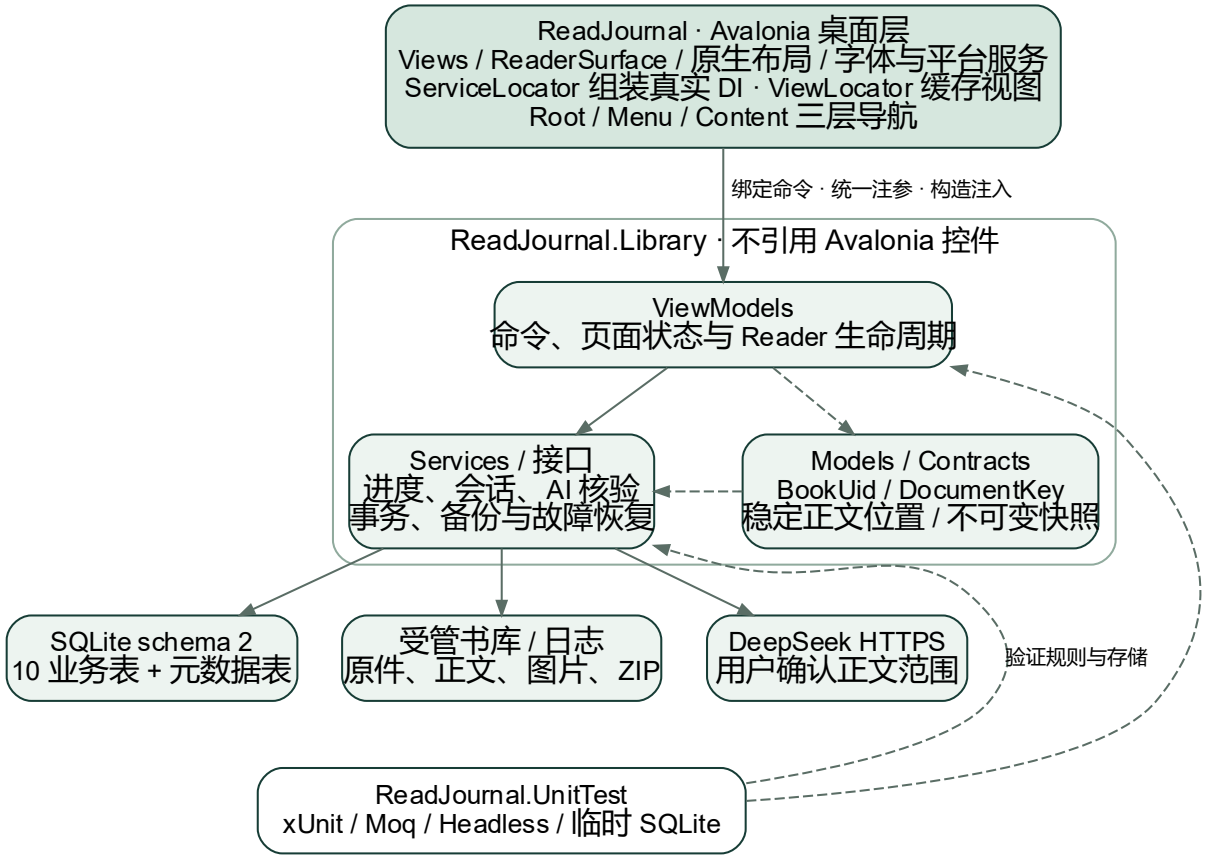


图 1 三工程分层、平台接口与外部资源依赖

ReaderSurface 与 AvaloniaReaderLayoutService 承担原生文本排版、绘制和选择命中测试，未引入 WebView 作为阅读方案。Library 使用正文位置、布局句柄和纯数据契约描述能力，具体排版由桌面层实现并经接口注入。实线表示主要运行依赖，虚线表示测试或数据契约协作。

课程架构要求与实际实现对应

要求	本项目实现
MVVM 分离	Views 负责展示及必要控件交互；ViewModels 持有命令和页面状态；Models/Services 负责规则和持久化。
统一参数注入	ViewModelBase.SetParameter 作为入口；内容导航校验参数后注入，参数版本隔离旧异步加载。
面向接口	存储、导航、正文、布局、会话、AI 和平台能力以接口定义；构造函数接收依赖，测试可替换实现。
真实 ServiceLocator	局部 ServiceCollection 注册单例，BuildServiceProvider 构建真实容器；只读 IServiceProvider 保存并解析，Current 从应用资源读取。
三层导航	Root 切换启动/主内容；Menu 清空返回栈；Content 压栈进入详情、编辑或阅读器；常量统一目标，MainViewModel 维护返回栈。
ViewLocator	IDataTemplate 匹配 VM，以约定名称反射创建视图、按类型缓存并设置 DataContext。
IO 服务化及异步	数据库、文件选择、正文 IO、HTTP、剪贴板、提示均由服务封装；异步命令提供忙碌、取消与错误处理。
UI/Library 双层	业务核心不绑定 Avalonia 控件；原生布局、字体目录和平台操作通过接口边界连接业务层。

2.2 功能模块设计

概览、书库、书摘、计划、统计及设置构成六个菜单入口。书库进入书籍详情、编辑或 ReaderView；详情关联阅读历史、计划和书摘；书摘可回到电子书的原文位置。编辑未保存时统一提供保存、放弃、取消；活动阅读器正常离开前须结束会话。左上角返回仅在存在上一级内容时显示。

运行期导航通过 NavigateToAsync 等待 MainViewModel.RequestNavigationAsync。同书原文回跳在现有 Reader 实例内执行；跨书、返回、菜单及关窗经统一离开编排，先取消或结束关联动作，再提交会话和切换内容。提交失败保留重试或继续阅读入口，避免页面已离开而会话未保存。

2.3 数据/数据库设计

数据库 user_version 为 2。原有四张业务表保留，增加电子书文件、进度、会话、书摘来源及 AI 批次数据，共 10 张业务表；另有 AppMetadata 元数据表。图中仅展示业务关系及代表字段，完整约束以 DatabaseMigrationV2 的建表 SQL 为准。

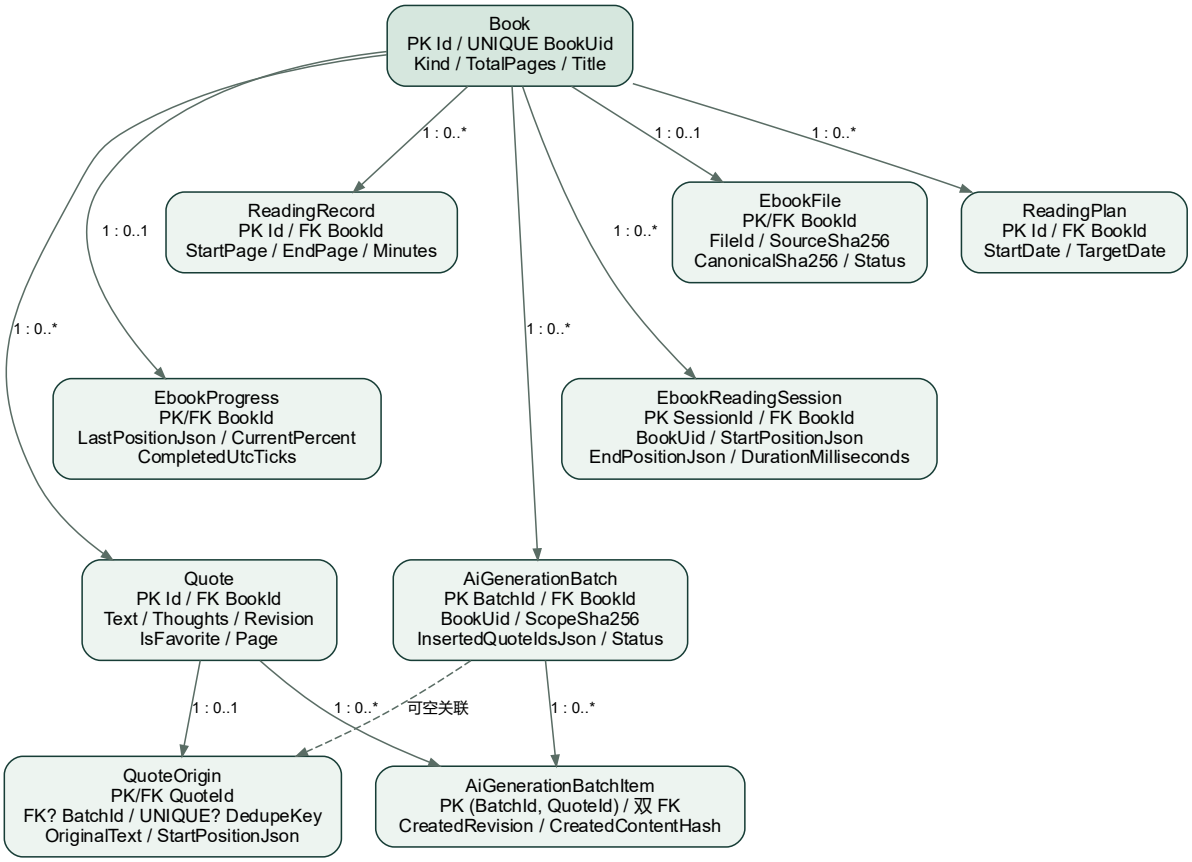


图 2 schema 2 的十张业务表及主要外键关系

实线外键默认 ON DELETE CASCADE；QuoteOrigin.BatchId 为可空关联，删除批次时 ON DELETE SET NULL。图中 0..1 表示可选的一对一扩展，0..* 表示可有多条子记录。AppMetadata 保存恢复事务标记，不参与业务实体关系图。

数据职责与关键约束

表/数据	关键字段与规则
Book	Id 主键，BookUid 唯一且不可变，Kind 区分手动与电子书；手动 TotalPages > 0，电子书 TotalPages = 0。
ReadingRecord	BookId 外键，仅用于手动书；起止页合法、分钟数为正，结束页不得超过总页数。
Quote / ReadingPlan	书摘保留 Text、Thoughts、收藏及 Revision；电子书书摘 Page 必须为空。计划关联书籍与起止日期。
EbookFile	BookId 一对一；FileId、来源/正文 SHA256、受管相对路径、ParserVersion 与 Ready/Missing/Invalid 状态。
EbookProgress	最后正文位置、CurrentPercent、HasStarted 与独立完成时间；重新打开恢复最后位置而非历史最高页。
EbookReadingSession	SessionId 主键；书籍/正文身份、起止位置、UTC 时刻与时区偏移、时长、关闭日及时钟异常标记。
QuoteOrigin	QuoteId 一对一；来源类型、原文与起止位置、上下文摘要、定位状态、可空批次关联和唯一去重键。
AiGenerationBatch	不可变批次回执含 ScopeSha256、插入 ID 清单、插入/重复/拒绝计数及可撤销状态。
AiGenerationBatchItem	BatchId + QuoteId 复合主键；生成时 Revision 与内容摘要用于判断撤销时是否仍可删除。

手动进度取有效记录的最大结束页，累计页数对每条闭区间长度求和并包含重读；电子书只统计正文百分比及会话时长。总时长先累加毫秒再换算，避免逐会话取整误差。完成标记与回看进度分离，重置仅清除当前进度和完成标记，保留历史与书摘；多个历史计划可能同时被重置为未完成时，整次操作拒绝。

迁移在事务内为旧书生成 BookUid，新增 Kind、Quote.Revision、业务表、索引和触发器；检查 integrity_check 和 foreign_key_check 后才写 user_version=2。旧字段不清空。触发器保护书籍身份、书籍类型、电子书零物理页数及批次回执不可变性，服务层仍承担完整业务校验。

2.4 接口与类设计

应用 UI 与 Library 包含 137 个类、41 个接口、134 个 C# 源文件和 15 个视图文件。按工程、命名空间及完整类型名合并 partial 声明，统计手写 class 和 interface，不计入测试工程、自动生成代码、record、struct 和 enum。

课程参考项目 DailyPoetryA 包含 63 个类、10 个接口和 9 个视图。按类与接口数量不少于参考项目 70%、视图数量不少于参考项目的要求，本项目达到相应规模。15 个视图文件包括窗口、初始化、容器和记录弹窗，其中业务页面为 11 个。

接口分组	职责与代表接口
基础存储与业务	IDatabaseService、IBookStorage、IQuoteStorage、IReadingService、IStatisticsService 等承担串行数据访问、事务和规则投影。
导航与平台	IRootNavigationService、IMenuNavigationService、IContentNavigationService；IFileService、IReaderPlatformService 等封装系统交互。
电子书导入	IEbookLibraryService、IBookParser，负责扫描/导入、TXT/EPUB 解析和删除协调；多个解析器按格式选择。
正文与排版	IReaderDocumentService 返回规范化正文与锚点；IReaderLayoutService 负责当前视口分页；IReaderFontCatalog 枚举本机字体。
阅读状态与摘录	IReaderSessionService、IReadingHistoryService、IReadingStateCalculator、IReaderQuoteService 统一会话、混合历史与原文定位。
AI 与凭据	IDeepSeekClient、IReaderAiService、IAiQuoteVerifier、IAiSettingsStorage、ISecretStore 分离请求、工作流、核验、配置与会话秘密。
数据维护与恢复	IApplicationDataCoordinator、ICompleteBackupService、IOperationRecoveryService 等管理租约、数据代际、完整恢复和失效清理。

ReaderOpenIdentity 同时携带 BookId、BookUid、DocumentKey、DataEpoch、ReaderInstanceId 和参数版本。整数 ID 可重用，不能单独作为异步写入依据；身份和租约均验证成功才允许提交。关键接口中的 CancellationToken 明确传播取消，数据契约使用 record 表示快照，但这些 record 不计入上述类数。

2.5 技术选型说明

技术/版本	选型与实际用途
.NET 8 / SDK 8.0.424	C# 桌面与业务核心；global.json 固定 SDK 补丁带并允许同补丁带更新。
Avalonia 11.0.10	跨平台桌面视图、主题及原生文本排版；实际验收以 Windows 为主，未测平台不承诺全面支持。
CommunityToolkit.Mvvm 8.2.2	可观察属性、命令与异步命令，减少界面状态通知样板代码。
Microsoft.Extensions.DependencyInjection 8.0.0	真实 ServiceCollection 容器、构造注入、单例及别名解析，保留课程参考组装方式。
sqlite-net-pcl 1.8.116	本地 SQLite 数据、ORM 映射及显式 SQL 迁移/触发器；数据库访问由应用协调器与串行门控制。
VersOne.Epub 3.3.6 / AngleSharp 1.8.5	读取 EPUB 内容结构并解析 XHTML；以受限规范化和原生绘制处理文本流，拒绝或降级超出范围内容。
HttpClient / System.Text.Json	真实 DeepSeek HTTPS、非流式 JSON；显式校验响应并保留供应商 usage，无自动错误重发。
xUnit / Moq / Avalonia.Headless.XUnit	业务规则、数据存储、异步竞态与真实绑定的自动化测试；结合实际窗口验证主要操作流程。

采用原生阅读方案便于复用 Avalonia 的文本测量、命中测试和主题体系，避免将业务坐标绑定到浏览器布局。代价是必须实现正文规范化、分页索引和稳定位置映射；因此首版限定为无 DRM 的流式 TXT/EPUB，不追求浏览器级 CSS 兼容。

依赖版本以工程文件和 packages.lock.json 为准，首次还原采用 --locked-mode。旧 SQLite 原生依赖可能产生 NETSDK1206 的旧 Alpine RID 提示；Windows 实测不据此推断 Alpine 或 macOS 支持。第三方依赖和许可元数据见配套源码 DEPENDENCIES.md、THIRD-PARTY-NOTICES.md；未声明许可的条目不能借用其他包结论。

3 系统实现

3.1 关键模块的核心实现

（1）真实容器、接口别名与导航

ServiceLocator 的构造函数集中登记服务与 ViewModel，全部按设计注册为单例。需要以多个接口共享同一实际对象时，先注册具体类型，再将接口映射到该具体实例；例如 FileService 同时提供普通文件服务与阅读器文件对话框能力。容器在注册结束后构建，属性从 IServiceProvider 解析。

代码 1 文件服务别名、容器构建与 VM 解析（实际节选）

```
        serviceCollection.AddSingleton<FileService>();
        serviceCollection.AddSingleton<IFileService>(sp => sp.GetRequiredService<FileService>());
        serviceCollection.AddSingleton<IAppFileDialogService>(sp => sp.GetRequiredService<FileService>());
        serviceCollection.AddSingleton<IReaderPlatformService, ReaderPlatformService>();
        serviceCollection.AddSingleton<IBackupService, BackupService>();
        serviceCollection.AddSingleton<ReadingPlanListViewModel>();
        serviceCollection.AddSingleton<ReadingPlanEditViewModel>();
        serviceCollection.AddSingleton<StatisticsViewModel>();
        serviceCollection.AddSingleton<SettingsViewModel>();
        _serviceProvider = serviceCollection.BuildServiceProvider();
    }
    public MainWindowViewModel MainWindowViewModel => _serviceProvider.GetService<MainWindowViewModel>()!;
    public InitializationViewModel InitializationViewModel => _serviceProvider.GetService<InitializationViewModel>()!;
    public MainViewModel MainViewModel => _serviceProvider.GetService<MainViewModel>()!;
```

源码：ReadJournal/ServiceLocator.cs，第 99—112 行。

内容导航首先校验目标和参数。ReaderOpenRequest 只能指向合法电子书及合法位置；运行期必须等待统一的异步离开编排，不能直接替换活动 Reader。相同书籍的原文回跳在现有会话中定位，避免创建第二条重复会话。

代码 2 同书原位定位与异步导航（实际节选）

```
public async Task<bool> NavigateToAsync(string view, object? parameter = null, CancellationToken ct = default)
{
    var target = Resolve(view, parameter); var main = ServiceLocator.Current.MainViewModel;
    if (target is ReaderViewModel reader && ReferenceEquals(main.Content, reader) && reader.IsSameBook(parameter!))
        return await main.RequestInPlaceAsync(() => reader.NavigateWithinBookAsync((ReaderOpenRequest)parameter!, ct), ct);
    if (ReferenceEquals(main.Content, target) && target is EditViewModelBase) return true;
    var accepted = await main.RequestNavigationAsync(() => NavigateTo(view, parameter), ct);
    if (accepted && ReferenceEquals(main.Content, target) && !target.OnLoadedCommand.IsRunning) await target.OnLoadedCommand.ExecuteAsync(null);
    return accepted;
}
```

源码：ReadJournal/Services/ContentNavigationService.cs，第 40—49 行。

（2）稳定正文位置与首屏生命周期

正文先解析为带章节与块信息的规范化表示，位置由 DocumentKey、ChapterKey 和 UTF-16 偏移构成；图片等非文本内容额外使用 NonTextBlockKey。DocumentKey 包含 FileId、来源摘要、规范化正文摘要和 ParserVersion，防止旧书摘被错误定位到另一版正文。字体、字号和窗口变化只改变布局，不改变原文位置。

代码 3　文档身份与可持久化正文锚点（实际节选）

```
public sealed record DocumentKey(
Guid FileId,
string SourceSha256,
string CanonicalSha256,
int ParserVersion);

public sealed record ReaderPosition(
DocumentKey Document,
string ChapterKey,
[property: JsonRequired] int Utf16Offset,
string? NonTextBlockKey = null);
```

源码：ReadJournal.Library/Models/Reader/ReaderContracts.cs，第 20—30 行。

打开阅读器依次验证书籍与数据代际、打开正文、解析已存位置并进入 Presenting。只有当前布局真正绘制并回调 PagePresentedAsync 后才进入 Active 并启动会话，避免加载失败或未显示页面也计时。视口变化替换首轮重排时保持呈现状态，不误判为 OpenFailed；真正退出或打开异常仍按取消/失败处理。

代码 4　首屏绘制确认后启动会话（实际节选）

```
public Task PagePresentedAsync(LayoutId layout)
{
    if (Phase != ReaderPhase.Presenting || _layout?.Id != layout || _identity is null || _position is null) return Task.CompletedTask;
    if (!_beginning.IsCompleted) return _beginning;
    return _beginning = Observe(async () =>
    {
        var identity = _identity; var position = _position; var now = _clock.Now;
        _context = await _sessions.BeginAsync(identity, position, now, _lifetime.Token);
        _start = position; _opened = now; Phase = ReaderPhase.Active; StartIndex();
    });
}
```

源码：ReadJournal.Library/ViewModels/ReaderViewModel.cs，第 121—131 行。

会话时长按关闭时刻减打开时刻计算，不扣闲置；时钟倒退时记录异常并将时长限制为非负。正常返回、菜单切换、切书和关窗使用冻结关闭命令，在同一事务内写入 SessionId 主键会话及最后位置，重复关闭查询已提交回执而不重复计时。强制杀进程不会补造未提交会话，只保证最后一次成功提交的数据。

（3）冻结 AI 范围与整段书摘

用户先选择当前页、当前选区或页范围，再执行解读或生成书摘。提交时冻结原文快照、正文身份、布局和范围摘要；响应后再次验证当前上下文，已退出、重置或恢复的数据不会接收旧响应。只有页范围模式显示起止页；当前页模式始终处理整页，不因存在高亮而悄悄变成选区。

选区生成书摘要求同一章节内连续纯文本、不跨图片且最多 8000 个 UTF-16 单元，并同时满足设置的输入上限。程序保留完整圈选原文，AI 仅返回整体解读，因此一段选择生成一条书摘，不再逐句拆分。当前页和页范围则可返回多条原文候选。

代码 5　选区原文由程序保留，模型只提供整体解读

```
        if (selection)
        {
            if (string.IsNullOrWhiteSpace(response.Analysis) || response.Analysis.Length > 4000)
                throw new AiException("InvalidAiResponse", "整段解读为空或超过 4000 字符，未保存书摘。");
            var span = scope.Original.Spans[0];
            result = verifier.Verify(scope, [new(span.Text, response.Analysis, null, null)]);
        }
        else result = verifier.Verify(scope, response.Candidates);
```

源码：ReadJournal.Library/Services/Ai/ReaderAiService.cs，第 62—69 行。

AiQuoteVerifier 使用 Ordinal 逐字查找，在冻结 TextSpan 中确认候选完整、连续、位于字符簇边界且能够唯一定位；原句重复时须有真实紧邻上下文消歧。找不到、歧义、越界或跨不连续片段拼接的候选明确拒绝，不能由模型返回的坐标直接入库。

代码 6　精确匹配、字符簇边界与紧邻上下文检查

```
            var boundaries = StringInfo.ParseCombiningCharacters(span.Text).Append(span.Text.Length).ToHashSet();
            var at = 0;
            while (at <= span.Text.Length - candidate.OriginalText.Length)
            {
                var found = span.Text.IndexOf(candidate.OriginalText, at, StringComparison.Ordinal); if (found < 0) break;
                var end = found + candidate.OriginalText.Length; at = found + 1;
                if (!boundaries.Contains(found) || !boundaries.Contains(end)) continue;
                exactOccurrences++;
                if (!string.IsNullOrEmpty(candidate.PrefixHint) && !span.Text[..found].EndsWith(candidate.PrefixHint, StringComparison.Ordinal))
continue;
```

源码：ReadJournal.Library/Services/Ai/AiQuoteVerifier.cs，第 21—29 行。

核验通过后，以事务保存 Quote、QuoteOrigin、批次项及不可变回执。去重键避免相同原文位置重复生成；本地保存失败仅重试已有批次，不重新请求模型。撤销只删除该批仍未改动、未收藏的新书摘，保留先前存在、用户编辑或收藏的内容。

（4）完整备份、操作日志与数据代际

旧 JSON 仅能描述手动业务数据，无法完整恢复电子书。当前完整 ZIP 包含业务快照、受管原件、规范化正文和图片资源清单；导出时在维护租约下检查数据库与受管文件，缺失文件库不能被当成完整备份。恢复入口仍接受旧 JSON，但会明确提示整体替换当前业务数据，包括电子书和会话。

代码 7　恢复前校验输入并生成、复核当前库自动备份

```
public async Task<RestoreResult> RestoreAsync(string path, CancellationToken ct)
{
    using var target = await Task.Run(() => _archive.Prepare(path, ct), ct); // Invalid input never changes the current data.
    await _database.InitializeAsync();
    await using var lease = await _coordinator.EnterMaintenanceAsync(MaintenanceKind.Restore, ct);
    await EnsureNoPendingRecoveryAsync(lease);
    var id = Guid.NewGuid(); var autoRelative = "backups/auto-backup-" + id.ToString("D") + ".zip";
    var automaticPath = ManagedBookFiles.SafePath(_paths.DirectoryPath, autoRelative);
    var old = _validator.Validate(await _database.ReadAsync(lease, BackupDataStore.Snapshot));
    var automatic = await ExportUnderLeaseAsync(automaticPath, lease, ct);
    using (var verified = await Task.Run(() => _archive.Prepare(automatic.Path, ct), ct))
        if (BackupDataStore.Hash(verified.Data) != BackupDataStore.Hash(old)) throw new IOException("自动备份复核失败，原数据未改变。");
```

源码：ReadJournal.Library/Services/Backup/CompleteBackupService.cs，第 57—68 行。

恢复先解压与校验目标，再写 Prepared 操作日志，将当前受管目录隔离到 trash、安装 staging 文件，并在 SQLite 事务内替换数据及写恢复标记。启动恢复根据持久化阶段与数据库标记判断回滚还是完成清理。导入和删除同样记录可恢复阶段；来源文件和收件文件不会随受管副本删除被移走。

维护操作阻止活动 Reader 与旧数据操作交错。替换成功更新 DataEpoch 并使旧页面和等待中的操作失效；无法完成对账时保留日志、staging、trash 和备份材料，显示 RecoveryRequired，避免通过静默清空掩盖不一致。

（5）字体与面板交互

字体目录枚举本机已安装字体，候选名称搜索、候选字体预览与中英混排预览共用有效字体选择。输入名称在点击应用前不会重排正文。默认或保存字体缺失时，仅在内存采用本机可用字体并提示，打开时不改写偏好；手动输入不存在的字体仍拒绝应用。

目录、排版与 AI 三面板独立收起，正文填充剩余空间；运行状态和取消入口在 AI 面板收起时仍可见。面板开关只在当前进程保留，重启恢复默认展开。F11 和按钮均可切换全屏，退出后恢复窗口状态；这些状态变化不替代正文锚点和会话状态。

3.2 核心功能与 UI 截图

以下界面展示书籍管理、原生阅读、AI 辅助和数据恢复功能。界面中的书籍与摘录均为自建测试样本，记录数量用于说明操作效果。

（1）阅读概览与业务联动



图 3 阅读概览：阅读指标、在读书籍、计划与近期记录

概览通过业务接口汇总书籍、记录和计划。手动页数来源于合法闭区间记录，电子书自动会话只累加时长；在读状态与完成状态由统一规则计算。侧边菜单提供固定业务入口，模块首页不显示无效返回按钮。

（2）书库、扫描与完成状态

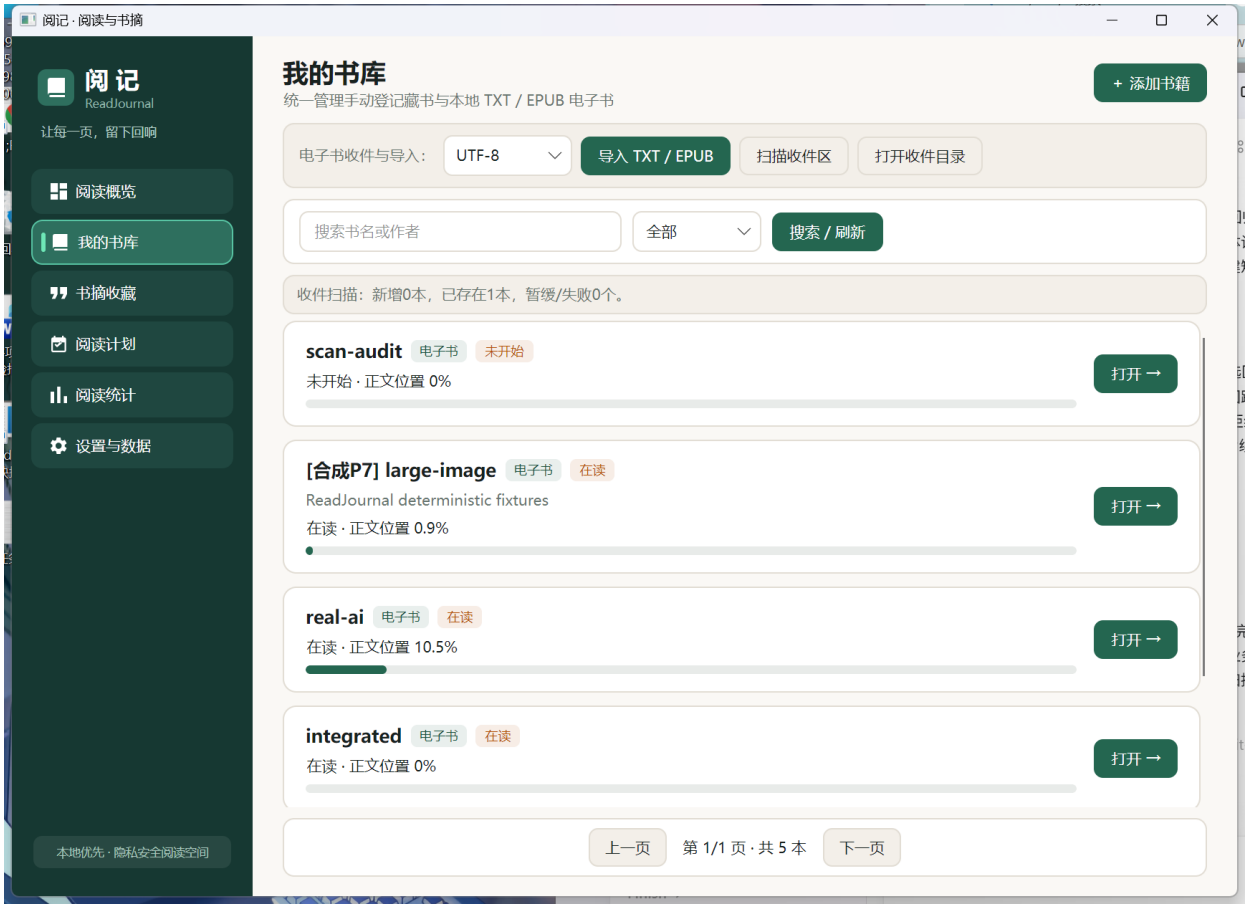


图 4　书库管理：手动与电子书共存，扫描入库与状态筛选

书库提供新增手动书、选择 TXT/EPUB、多次扫描收件区、搜索和状态筛选。导入时复制来源文件到受管目录，按来源 SHA256 去重。同一本书导入两次不会创建两册；扫描再次发现相同文件也不会重复入库。

重置阅读进度后，系统回到首章偏移 0，清除完成标记，并保留历史记录和书摘。删除书籍后，若来源文件仍在收件区，系统不会通过扫描自动重建该书籍；用户可通过文件选择器主动重新导入。

（3）原生正文选择与书摘定位

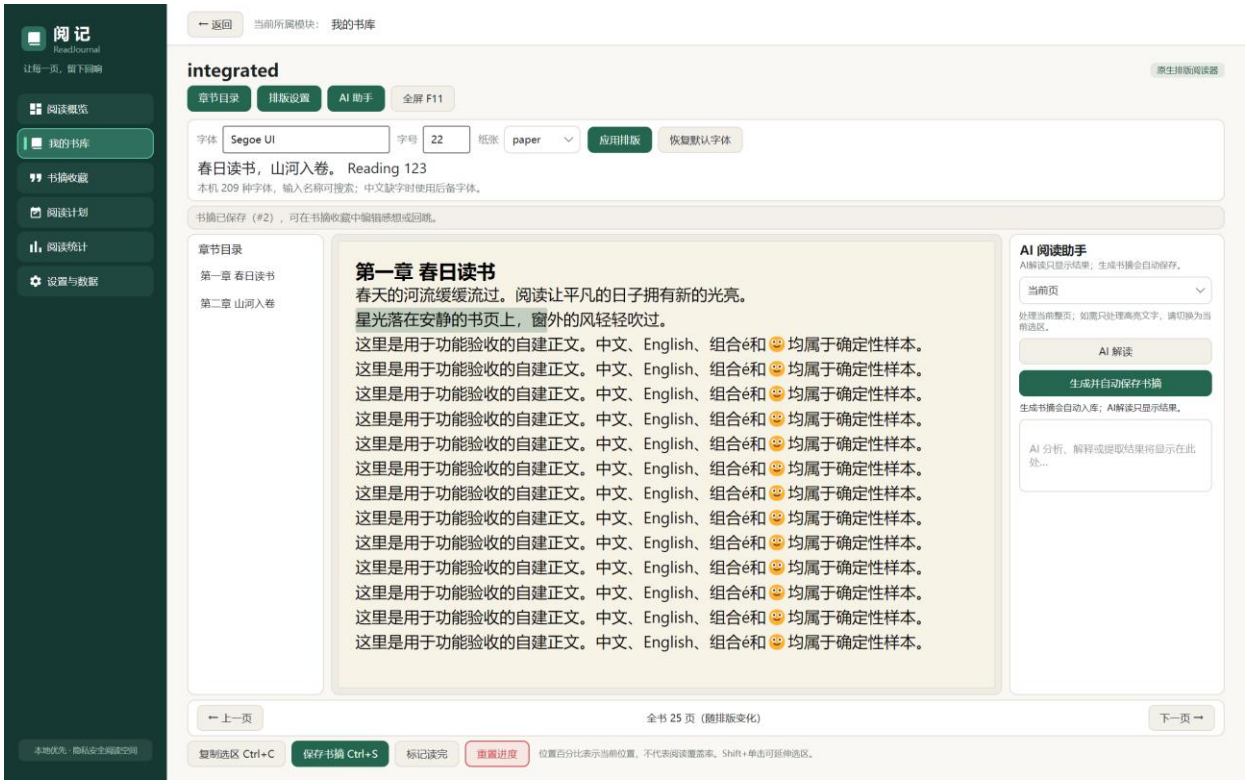


图 5 原生阅读器：目录、字体排版、正文选择及书摘快捷键

正文使用原生排版表面，不是网页截图。物理拖选可复制，Ctrl+S 将有效原文位置、原文内容和来源信息保存为书摘；同一位置再次保存可去重。书摘列表中的原文入口能恢复对应书籍、章节和位置，复制回跳高亮区得到的文字与原文逐字一致。

目录跳转、左右键或 PageUp/PageDown 翻页、字体/字号变化和窗口重排均通过正文锚点衔接。字体输入具有搜索和预览，只有点击应用才改变排版并持久化；不存在的字体会明确拒绝，不破坏原设置。

（4）面板收起与沉浸阅读

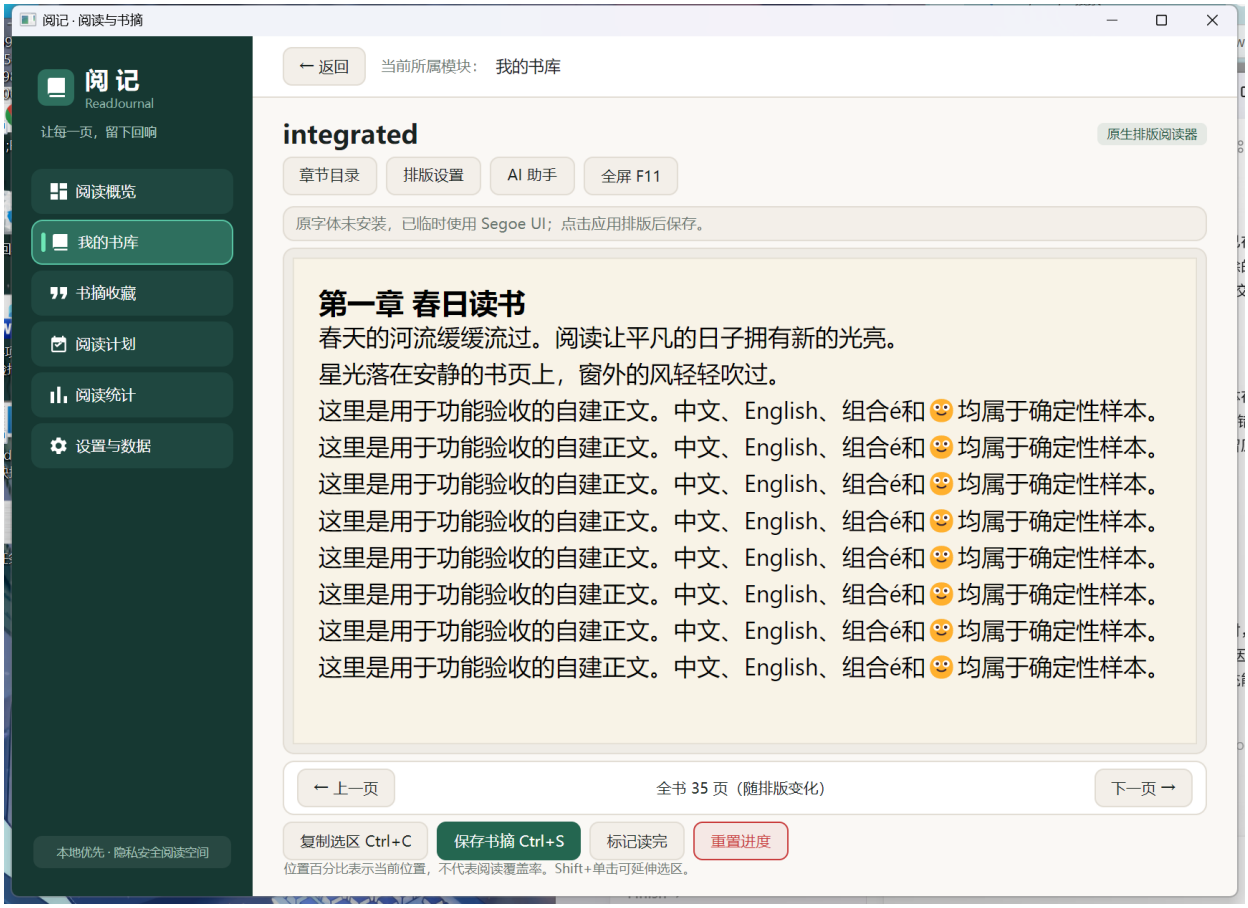


图 6 目录、排版与 AI 面板收起后，正文填充可用区域

面板收起后，顶部开关和全屏入口仍可使用，正文选择及翻页继续有效。三个面板可分别恢复；界面尺寸变化会触发新布局，稳定正文位置保持不变。AI 任务运行时，即使助手面板被收起，顶部仍显示状态和取消入口。

操作	行为
F11 / 全屏按钮	切换全屏，退出后恢复窗口状态。
左右键 / PageUp / PageDown	在当前有效阅读布局中翻页。
Ctrl+C / Ctrl+S	复制当前原生选区 / 保存可定位书摘。
字号或窗口变化	重新排版并保留正文锚点；旧页范围须重新填写。

（5）真实 AI 与整段选区书摘



图 7 真实 DeepSeek：选区整体解读并将完整原文保存为一条书摘

选区书摘以完整原文为一条记录，AI 提供整体解读。测试中，选定的 9 字原文完整保存，重复生成未增加重复书摘；从书摘回到原文后复制内容一致，撤销生成后恢复到操作前的书摘数量。

当前页和页范围可以生成多条候选，每条均经连续原文核验；单独执行“AI 解读”只显示结果，不写入书摘。应用通过 DeepSeek 接口获得响应，测试工程通过模拟 HTTP 处理器覆盖错误、取消及迟到响应等情况。

截图展示成功路径，不表示每次模型输出都会通过校验；结果区区分部分成功、全部拒绝及无新增，用户仍须判断解读内容。

（6）EPUB 原生图片与稳定续读

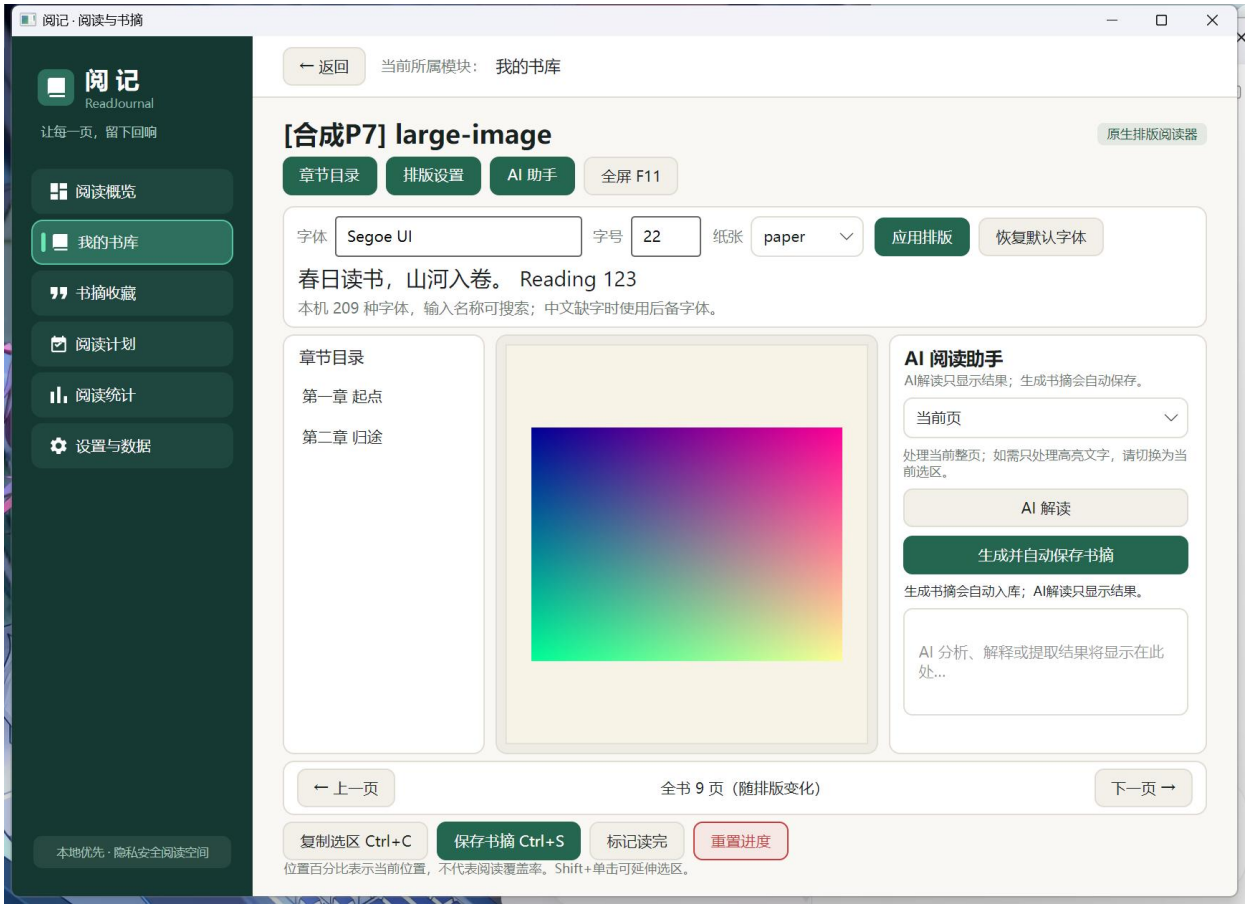


图 8 自建 EPUB 的 2400×1800 图片在原生阅读表面显示

EPUB 从内容结构与 XHTML 解析为章节、段落、标题、基础样式及本地图片块。受检图片按视口显示，非文本块不充当可引用原文，跨图片的内容不会被拼成一条“连续原句”。

关闭并重新打开带图 EPUB 后，系统可根据非空 NonTextBlockKey 恢复图片所在位置。图片锚点与文字锚点共同提供稳定定位，动态页号只用于当前布局，避免字体或字号变化后续读错位。

截图中的颜色图块来自确定性自建 EPUB，用于验证大图显示和重启定位，不代表已覆盖商业出版物的复杂排版。

（7）主题、真实 AI 配置与备份入口



图 9 深色设置页：主题、完整备份恢复与真实 AI 配置

设置页集中管理主题、导出/恢复、上传知情及真实 AI 配置。凭据在隐藏输入框中输入，保存、错误或离开后清空输入框；删除会话 Key 后再次调用会在 HTTP 前明确拒绝，但本地阅读与正常会话保存不受影响。

业务恢复不覆盖当前机器的主题、AI 配置和会话 Key。建议导出完整 ZIP，而不是直接复制正在写入的 SQLite 文件。旧 JSON 恢复兼容手动书籍，但不能承诺保留现有电子书资源；界面在操作前说明整体替换语义。

3.3 难点和解决方案

实际问题	处理与验证
动态页号无法用于长期定位	规范化正文配合 DocumentKey、章节和 UTF-16 锚点；图片使用非文本块锚点。重排后解析新页，永久回归与实际重启/回跳验证。
手动页数与电子书进度冲突	Kind 区分书籍；电子书 TotalPages 为 0 且不生成手动页记录；统一状态计算器汇合时长、计划与完成状态。
缺失字体阻断排版	打开时采用临时可用字体并保留原偏好，点击应用才保存。测试确认缺失字体时仍可改字号，手动错误字体被拒绝，默认字体可恢复。
首屏重排取消导致会话不激活	呈现前先进入 Presenting；视口替代取消不标成 OpenFailed；仅实际绘制启动会话。可控取消回归和真实窗口均验证只保存一条会话。
AI 书摘改写、碎片化或重复	整段选区原文由程序保留；其他候选逐字连续匹配及消歧；数据库去重、原文回跳和撤销形成闭环。
迟到响应与数据恢复交错	请求冻结快照，响应后重验身份、会话和 DataEpoch；事务写入再次校验，旧响应不能写入已恢复或已重置的数据。
文件替换与数据库无法共用事务	通过 staging/trash、持久化阶段日志与数据库恢复标记编排；实际进程中断后启动对账，不清空坏日志掩盖失败。

AI 本地落库失败与远端生成失败分开处理：前者保留当前会话内有界待保存批次，以同一回执重试；后者明确显示错误，不自动重发。事务回滚和提交确认丢失均有回归，避免重复收费或重复插入。

已知能力边界包括复杂 CSS、固定版式、DRM、非 UTF-8 XML、SVG/公式/音视频等不支持或明确降级内容。长选区跨章节、跨图片或超过限额时要求缩小范围，不自动改写成不可验证的摘录。

4 系统测试

4.1 测试环境

项目	环境与方法
实际窗口环境	Windows 11 x64，实际 144 DPI / 150% 缩放；SDK 8.0.424，.NET 运行时 8.0.30。
测试构建	Release；UI、Library、UnitTest 三工程；xUnit、Moq、Headless 视图及临时真实 SQLite。
样本与隔离	自建 TXT、百万字符文本、确定性 EPUB 及旧 schema1 夹具；使用独立数据目录，不修改用户书库。
测试日期	2026 年 10 月 24 日至 25 日；界面功能测试与源码包独立构建测试。
构建与运行	独立解包后使用 --locked-mode 还原依赖，Release 构建并运行自动化测试；启动方法见源码包 README。
测试边界	100%/200% 与跨屏、真实商业 EPUB、未测试平台；2 项 Windows 符号链接测试因权限跳过。

测试结果汇总

检查	结果
完整自动化回归	共 353 项：351 通过、0 失败、2 项因符号链接权限跳过。
架构集成与性能测试	153 项通过，覆盖 15 个视图模型和 36 个服务接口；反复切书后文档、布局和解码图片引用归零。
原生排版实验	14 / 14 通过。
EPUB 样本	11 个支持/降级变体、15 个异常拒绝变体通过；商业样本 0，未测市场兼容率。
AI 功能与数据恢复	解读、书摘生成、去重和撤销验证通过；恢复前后 10 张业务表记录及 12 个受管文件摘要一致。

两项跳过测试涉及受管内容、目的地和来源/收件区符号链接，原因是 Windows 账号没有创建符号链接的权限，因此不计入通过数量。自动化测试、架构集成测试、排版实验及界面测试采用不同方法，结果分别统计。

4.2 测试用例

功能测试在独立数据目录中进行，覆盖正常输入、非法输入、重复操作、异常响应及关闭重启等情况。下表列出各场景的输入、预期行为和实际结果。

编号/场景	输入与预期	实际结果
T01 手动业务	空书名/非法页码拒绝；合法书籍、记录、书摘及计划入库。	符合；实际窗口与 SQLite 快照一致，收藏切换可持久化。
T02 混合统计	手动页码按闭区间累计；电子书只计时长/百分比，计划与概览联动。	符合；混合统计和永久回归无虚构电子书页数。
T03 TXT 去重/扫描	同文件重复导入及重复扫描，不增加重复书籍；来源保持。	符合；选择器导入两次仅一册，新增收件文件扫描成功。
T04 原生选择	物理拖选、复制、保存与再次保存，回跳后复制应一致。	符合；保存去重且原文逐字一致。
T05 重排/面板	改变字号/视口，开关面板和全屏，应保持可操作及稳定位置。	符合；三面板及全屏实际窗口验证，锚点/选区由回归覆盖。
T06 缺失字体	默认或保存字体不存在，仍可改字号；手动错误字体拒绝。	符合；两个缺失字体回归及 209 种字体目录实际窗口验证。
T07 首屏竞态	首轮排版被新视口替换，仍呈现并激活，退出只存一会话。	符合；稳定复现回归修复后通过，真实窗口正常翻页/保存。
T08 EPUB 续读	导入带图 EPUB，关闭于图片，重启应恢复非文本块位置。	符合；2400×1800 图显示，非空图片锚点与重启截图一致。
T09 完成/重置	完成后回看不取消完成；重置回首章并保留历史与书摘。	符合；完成标记清除、偏移归零、重新打开仍可激活。
T10 旧库迁移	schema1 迁到 2，旧四类实体字段保持，第二实例不能写。	符合；迁移后原有字段保持，重复实例写入被阻止。

AI、恢复与故障用例

编号/场景	输入与预期	实际结果
T11 解读不入库	当前页或页范围真实解读，书摘数量应保持。	符合；当前页 86 字、页范围 1～1 页请求成功。
T12 选区整段书摘	9 字选择生成一条，AI 只解释；重复生成不重复。	符合；原文精确保留，第二次没有新增重复书摘。
T13 页内候选核验	当前页生成多条，必须逐字连续匹配并可唯一定位。	符合；新增 2 条全部匹配；伪造、歧义、跨片段由回归拒绝。
T14 原文回跳/撤销	回跳复制应一致；撤销仅删除本批未改动的新书摘。	符合；回到原有 2 条；编辑、收藏、原有重复保护由回归覆盖。
T15 错误/凭据	TCP 拒绝或删 Key 后仍可读、可正常结束；无自动重发。	符合；真实拒绝连接与删除 Key 入口验证，HTTP 异常/取消回归通过。
T16 旧响应/保存重试	退出/恢复/重置后旧响应拒绝；本地重试不增加 HTTP。	符合；身份、epoch、回执及事务回归通过。
T17 完整恢复	导出→删 EPUB→恢复→重启→回跳；业务和受管资源一致。	符合；10 张业务表内容、12 文件摘要一致，本机主题保留。
T18 中断恢复	备份恢复 4 处、TXT 导入/删除 10 处实际中断后对账。	符合；各中断位置均能恢复对账；损坏日志保留供恢复，不静默清除。

性能测量与范围说明

性能测试运行于 Windows、.NET 8.0.30 和 32 逻辑处理器环境。使用生产 ReaderViewModel 与 ReaderSurface 进行 Skia 离屏绘制，视口为 760×440 DIP、输出 1140×660 像素。测量区分冷导入、预热打开、翻页和重排，不将离屏采样等同于物理显示缩放测试。

指标	测量结果	解释
百万字符冷导入	约 2120.64 ms	解析/复制导入成本，独立于后续打开与首绘。
预热打开 + 离屏首绘	约 465.10 ms	已做路由预热，OS/磁盘/字体缓存已热；非全新环境冷启动首屏。
200 次翻页 P95	约 6.68 ms	同一组测试的翻页耗时分布；不外推所有设备。
20 次重排 P95	约 129.87 ms	同一组测试中字号/布局变更采样。
布局调度最大间隔	约 43.21 ms	UI Dispatcher 心跳，不等同于物理键鼠输入延迟。
20 次切书后引用	文档/布局/解码图片均 0	生命周期引用清理结果，不等价于操作系统内存立即归零。

冷导入与预热首绘分别反映解析导入和缓存预热后的打开成本，不合并为同一首屏耗时。P95 描述本组有限样本的分布，结果会受设备、书籍、字体和缓存状态影响。

4.3 测试截图

图 10 展示完整恢复并重启后的阅读界面。测试流程为导出备份、删除电子书及关联数据、恢复备份、关闭并重启应用，再检查内容和原文定位。

恢复前后对比 10 张业务表中的记录及 12 个受管文件的 SHA256 摘要，结果一致。恢复后，本机深色主题保持，受管电子书、图片和书摘位置仍可使用。



图 10 恢复并重启后的 EPUB 仍可阅读，主题与原文能力保持

恢复并重启后，再次打开 EPUB，正文、图片和书摘原文定位均可使用。结合选区复制、书摘去重和批次撤销测试，验证了界面、数据库和受管文件之间的协作。

5 总结与展望

阅记已从阅读活动管理扩展为原生 TXT/EPUB 阅读与 AI 辅助应用，保留手动书籍、阅读记录、书摘收藏、计划和统计等原有功能。通过真实 DI、三层导航和 UI/Library 分层，将原生排版与平台操作连接到可测试的业务核心；通过稳定正文锚点、幂等会话、逐字原文核验和完整 ZIP 恢复，使新增阅读能力与原有数据闭环衔接。

源码包在独立目录中完成锁定依赖还原、Release 构建和自动化测试。353 项测试中 351 项通过、0 项失败、2 项因符号链接权限跳过。界面测试覆盖主要阅读流程，备份恢复后 10 张业务表与 12 个受管文件一致，支持系统在已测试环境中的可用性。

系统实现的重点是保持数据语义与界面状态一致：手动页码和电子书百分比分开计算，阅读器通过稳定正文锚点适应重排；统一导航在切换页面前结束活动会话，事务与操作日志共同处理数据库和文件恢复。AI 仅处理用户确认的范围，原文核验和本地重试降低了错误摘录及重复请求的风险。

后续优先补充 Windows 100%/200% 与跨屏实测、具备权限环境下的符号链接用例和授权商业 EPUB 样本，再逐步扩大复杂排版兼容及大书库压力测试。AI 方面继续改善解读可读性与拒绝原因提示，保持原文可验证、范围明确及本地保存重试不重复请求的边界。账号、同步与在线书源不在当前实现范围。